

Setup für einen ‚privaten‘ „Copilot“ in VS Code

Quellen, welche Extensions für AI Autocomplete/ Chat zur Verfügung stehen:

https://www.perplexity.ai/search/welche-visual-studio-code-exte-Xo._f4Y7QuiNuKdwqLEjqQ

https://www.perplexity.ai/search/recherchiere-nach-aktuellen-mo-b_RS0duBQ3aroMPPe4KKXg

Auswahl: Continue

a) Die einzige Extension, die auch ohne Login funktioniert, ist 'Continue'. Außerdem ist sie OpenSource.

<https://www.continue.dev/>

In Visual Studio Code sieht das so aus:



ToDo: Continue-Extension in VS Code installieren.

b) Die einzige Methode, die 100%ig sicher ist: Ein lokales Sprachmodell, am besten ein Coding-Modell. Als Backend-Server auf dem eigenen Computer nutzen wir Ollama:

<https://ollama.com/>

ToDo: Installationsdatei herunterladen & installieren (läuft bei Mac und Windows gleich ab)

c) Für die Autocomplete-Funktionalität lädt man innerhalb von Ollama zum Beispiel dieses Modell hier:

	Modell-Link	Befehl für die Eingabe-Aufforderung:
Optimal, aber groß	https://ollama.com/library/qwen2.5-coder:14b-base	ollama run qwen2.5-coder:14b-base
Alternativ, Mittelgross	https://ollama.com/library/qwen2.5-coder:7b-base	ollama run qwen2.5-coder:7b-base
Alternativ, Klein	https://ollama.com/library/qwen2.5-coder:3b-base	ollama run qwen2.5-coder:3b-base

ToDo: in die Eingabe-Aufforderung (Windows) oder Terminal (Mac) den Befehl ausführen

Screenshot Windows: Eingabeaufforderung	Screenshot Mac: Terminal

OLLAMA CHEATSHEET: Nützliche Befehle

Quelle:

https://www.perplexity.ai/search/erstelle-die-wichtigsten-befeh-XBC_sTVFRDOB.KeL4g5Z6A

Links mit vertiefenden Infos:

<https://www.glukhov.org/post/2025/05/how-ollama-handles-parallel-requests/>

<https://www.arsturn.com/blog/advanced-configuration-settings-for-ollama>

Zweck	Befehl (Beispiel)
Modell laden	ollama pull llama3
Modell löschen	ollama rm llama3
Installierte Modelle anzeigen	ollama list
Modell-Details	ollama show llama3
Modell starten	ollama run llama3
Aus ollama-chat nach laden aussteigen	/bye
Laufende Modelle anzeigen	ollama ps
Modell stoppen	ollama stop llama3
Modell konfigurieren	ollama run llama3 /set parameter num_ctx 2048
Hilfe	ollama help
Informationen über ein Modell	ollama show llama3
Hilfe zu bestimmten Befehlen	ollama list --help ollama ps --help ollama show --help usw.
Versionsnummer zeigen	ollama --version

d) Für die Chat-Funktion innerhalb von VS Code kann man ein DeepInfra-Modell nehmen. DeepInfra ist aktuell der einzige Inference-Provider, der zusagt, keinerlei Prompts zu speichern. Sehr gut geeignet ist dieses Modell:

<https://deepinfra.com/Qwen/Qwen2.5-Coder-32B-Instruct>

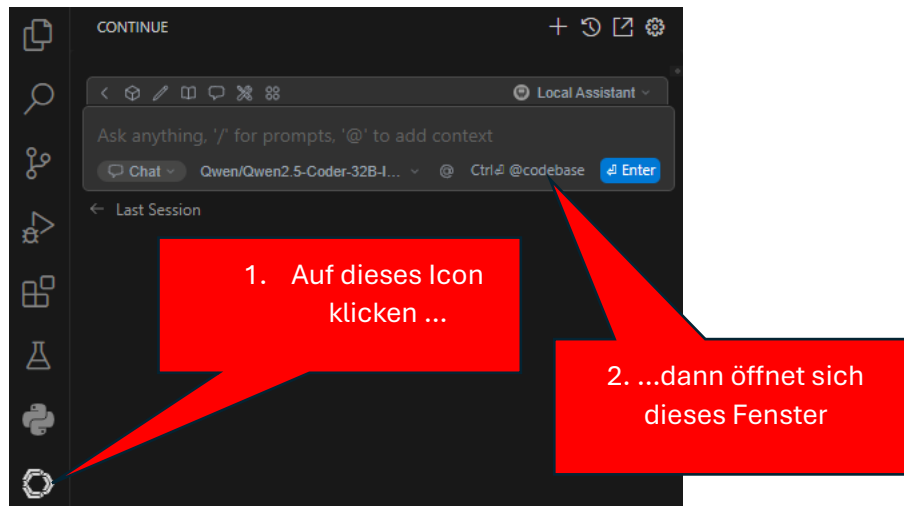
Dieses LLM gilt als eines der derzeit besten Coding-LLMs:

(Quelle: <https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard>)

ToDo: notwendige Informationen für den späteren Konfigurations-Eintrag in VSCode sammeln

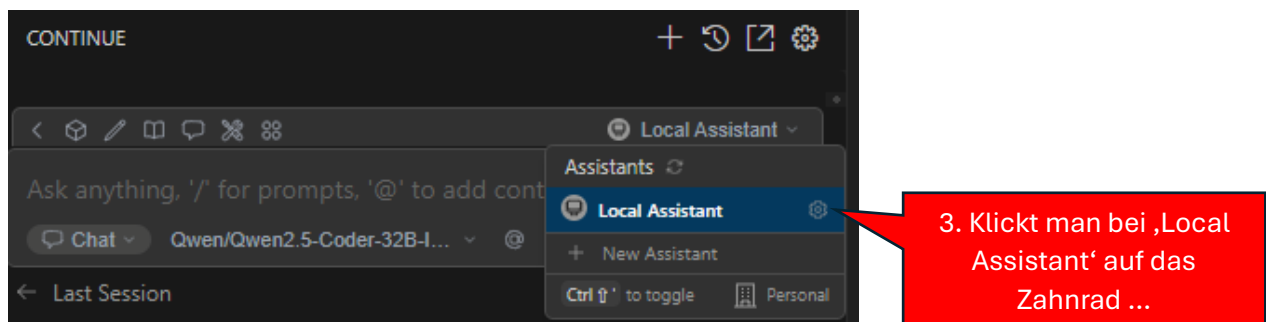
e) Nun zurück in VS-Code.

ToDo: Auf das Continue-Icon für die Konfiguration klicken:



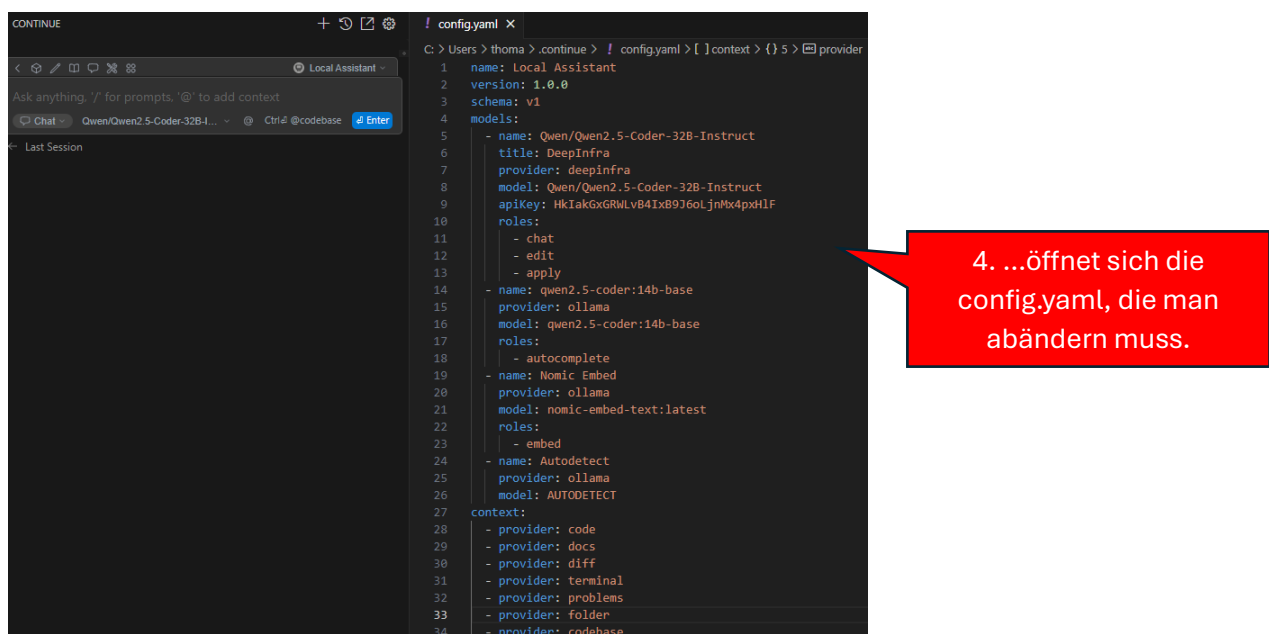
f) Modelle konfigurieren.

ToDo: bei ‚Local Assistant‘ auf das Zahnrad klicken



g) config.yaml (bzw. nächste Seite für Mac: config.json) anpassen.

ToDo: Daten von DeepInfra und Ollama bereithalten und korrekt eintragen



ACHTUNG, MACOS: Dort wird aktuell noch die „config.json“ geöffnet; dort ist aber in kleiner Schrift oberhalb des Datei-Textes zu lesen „config.json is deprecated, convert to config.yaml“. Dort draufklicken, dann wird die Datei konvertiert!

Muster für config.yaml

```
name: Local Assistant
version: 1.0.0
schema: v1
models:
  - name: Qwen/Qwen2.5-Coder-32B-Instruct
    title: DeepInfra
    provider: deepinfra
    model: Qwen/Qwen2.5-Coder-32B-Instruct
    apiKey: HkIakGxGRWLvB4IxB9J6oLjnMx4pxHlF
    roles:
      - chat
      - edit
      - apply
  - name: qwen2.5-coder:14b-base
    provider: ollama
    model: qwen2.5-coder:14b-base
    roles:
      - autocomplete
  - name: Nomic Embed
    provider: ollama
    model: nomic-embed-text:latest
    roles:
      - embed
  - name: Autodetect
    provider: ollama
    model: AUTODETECT
context:
  - provider: code
  - provider: docs
  - provider: diff
  - provider: terminal
  - provider: problems
  - provider: folder
  - provider: codebase
```

Quelle: ChatGPT Recherche:

<https://chatgpt.com/share/68337bdd-4cac-800f-91d3-801364149f88>

Wenn alles funktioniert hat, öffnet man das Continue-Fenster

... und kann losarbeiten.

Grundsätzlich gilt:

- Für das Autocomplete benötigt man ein ‚base‘-Modell, welches NICHT auf die Antwort auf Befehle hin trainiert wurde.
- Für den Chat benötigt man ein ‚instruct‘-Modell, welches auf Nutzer-Befehle trainiert wurde.

1. Autocomplete mit dem Ollama-Modell:

„#“ eingeben PLUS kurzen Kommentartext. Nach einigen Sekunden antwortet **Autocomplete**

2. Chat nutzen:

Hier den Befehl eingeben und mit ‚Enter‘ bestätigen.

Hier den Quellcode kopieren und ins Jupyter-Notebook übernehmen.

Addon-Information zu DeepInfra:

Anonymität:

Mittel. Es ist ein Account (via GitHub oder E-Mail) und in der Regel eine Zahlungsmethode erforderlich, was die Nutzung an eine Identität bindet. Die Inhalte der Anfragen sollen jedoch geschützt sein.

Datenspeicherung & Logging:

Explizite Aussage (Blog, April 2025): DeepInfra wirbt mit einer "**strikten No-Logging-Policy für User-Prompts**". Dies ist die stärkste Datenschutzzusage unter den verglichenen Anbietern.

Drittanbieter-Apps: Integrationen (wie WatchWolf) bestätigen, dass DeepInfra keine Anfragen/Antworten speichert, die über sie laufen (Stand Jan 2025).

Standard-Datenschutzrichtlinie (Stand Juli 2024): Enthält übliche Klauseln zur Datennutzung für rechtliche Zwecke, betont aber auch, dass keine Daten an Dritte verkauft oder weitergegeben wurden und dies auch nicht geplant ist.

AGB/Datenschutzrichtlinie: Die öffentliche Zusage zur "No-Logging-Policy" ist ein starkes Signal, auch wenn die detaillierten AGBs dies noch genauer spezifizieren müssten. Es bleibt eine gewisse Unklarheit, ob "no-logging" auch temporäre Speicherung für Abrechnung oder Fehleranalyse ausschließt.

Sicherheitsstandards:

DeepInfra erfüllt verschiedene Sicherheitsstandards, darunter SOC 2, ISO 27001, HIPAA und GDPR. Dies deutet auf ein hohes Maß an Sicherheitsbewusstsein hin.

Fazit:

Wenn der Schutz des *Inhalts* Ihrer Anfragen oberste Priorität hat und Sie bereit sind, einen Account zu nutzen, scheint **DeepInfra** aufgrund seiner expliziten "No-Logging"-Policy die beste Wahl zu sein.

Für maximale Anonymität und Datensicherheit bleibt die Nutzung von **lokalen LLMs** auf Ihrer eigenen Hardware auch im Jahr 2025 die sicherste Methode.

Quelle: GPT 4.5 & Gemini Deep Research:

<https://g.co/gemini/share/edbf074ccda5>

<https://chatgpt.com/share/68337c6c-8d3c-800f-a43b-3d14672ffc58>